

How Your ROS 2 System Works

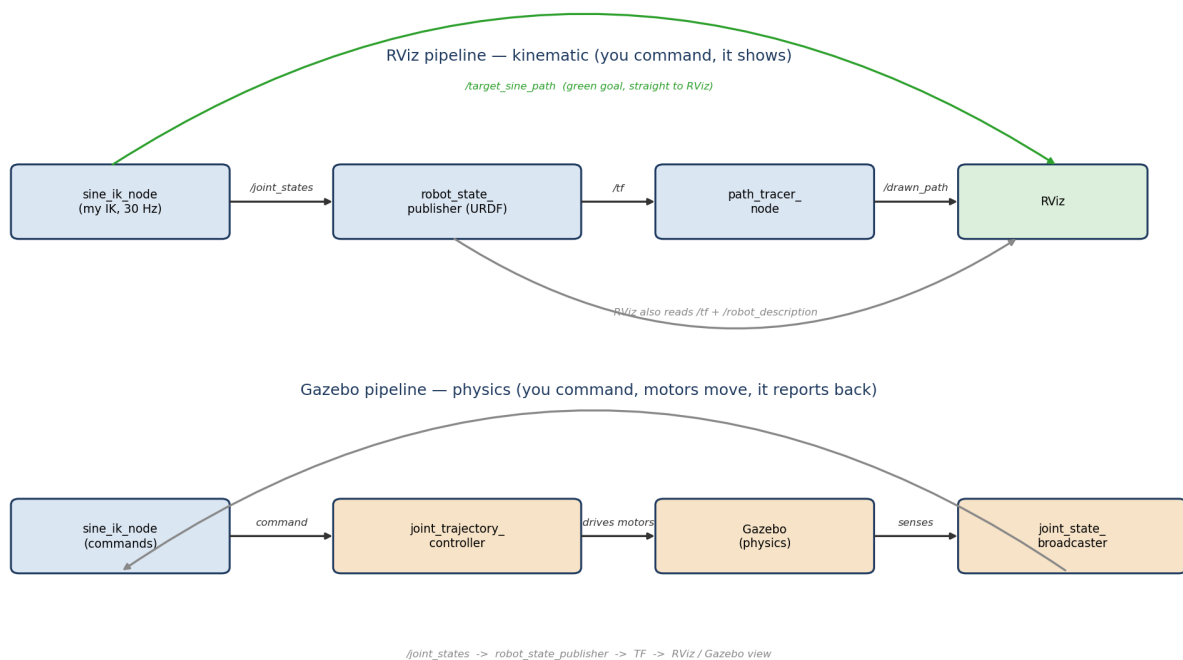
R_sine — a plain-language guide to the nodes, topics, TF, and the difference between the RViz and Gazebo pipelines.

The 3 core ideas

- **Node** = one small program doing one job. This project runs three: `sine_ik_node`, `robot_state_publisher`, `path_tracer_node`.
- **Topic** = a named channel that nodes post to or read from (like a radio channel with a fixed name and a fixed message format).
- **Message type** = the shape of the data on a topic, e.g. `/joint_states` carries a `sensor_msgs/JointState` (joint names + angles).

Nodes never call each other directly: a **publisher** posts to a topic and any **subscriber** on that topic receives it. For `/joint_states` there is 1 publisher (`sine_ik_node`) and 1 subscriber (`robot_state_publisher`).

The data flow (RViz pipeline)



```
[sine_ik_node] --/joint_states--> [robot_state_publisher] --/tf--> [path_tracer_node]
(my IK: 6 angles                    (has the URDF; converts          (reads base_link->
at 30 Hz)                          angles -> pose of every         link6_flange,
|                                  link, as TF frames)             publishes /drawn_path)
|                                  |                                |
+----- /target_sine_path (green goal) -----+                    v
|                                                                 [ RViz ] <-- /tf, /robot_description,
|                                                                 both path topics
```

Step by step

- **sine_ik_node** precomputes the inverse kinematics once, then publishes the 6 joint angles to `/joint_states` 30 times per second. It also posts the green target sine to `/target_sine_path`. It only talks, never listens.
- **robot_state_publisher** is given the URDF (the robot geometry) once and subscribes to `/joint_states`. For each new set of angles it computes where every link ends up and publishes that as TF.
- **TF (/tf)** is the tree of coordinate frames — the position and orientation of `base_link`, `link1 ... link6_flange` relative to one another. It is a second, independent forward kinematics (done from the URDF), which cross-checks ours.
- **path_tracer_node** subscribes to `/tf`, asks 'where is `link6_flange` relative to `base_link`?', and appends that point to `/drawn_path` — the red drawn line.
- **RViz** subscribes to `/robot_description` (draws the mesh), `/tf` (poses it), and the two path topics (green target, red drawn).

The launch file (`rsine_sine.launch.py`) simply starts these processes, hands the URDF to `robot_state_publisher`, and passes the sine parameters to `sine_ik_node`. One launch = the whole graph above.

Why Gazebo is different

In the RViz pipeline WE choose the joint angles (our IK) and simply announce them — nothing pushes back. It is kinematic: no gravity, motors, or contact. Gazebo simulates physics, so you cannot teleport joints; you COMMAND them and a controller drives the motors while physics responds. That is what `ros2_control` provides:

- **controller_manager** — the conductor; loads and runs controllers.
- **hardware interface** — what controllers talk to. In Gazebo it is `gz_ros2_control` (the simulated motors); on the real arm it is the serial driver. Same controllers, swap the hardware.
- **joint_trajectory_controller** — takes a timed trajectory and drives the joints to it smoothly.
- **joint_state_broadcaster** — reads the ACTUAL joint positions back from the sim and publishes `/joint_states` (the reverse direction from the RViz setup).

So in Gazebo the flow flips: `sine_ik_node` -> controller command -> (physics) -> `joint_state_broadcaster` -> `/joint_states` -> `robot_state_publisher` -> TF -> RViz/Gazebo. Two things must be added: a controller config (YAML) and `ros2_control` tags in the URDF marking which joints have motors.

Key mental model: RViz shows what you command; Gazebo shows what physically happens when you command it.